2026

UNIVERSIDAD TÉCNICA DE MANABÍ
FACULTAD DE CIENCIAS INFÓRMATICAS
TECNOLOGÍAS DE LA INFOMRACIÓN

DESARROLLO DE SISTEMAS INFORMATICOS

ACTIVIDAD #1: ESTUDIO DE CASO - SISTEMA DE CONTROL DE RESCATE Y ADOPCIÓN ANIMAL

.

NOMBRE: MARLON ANDRÉS ACUÑA MONTERO

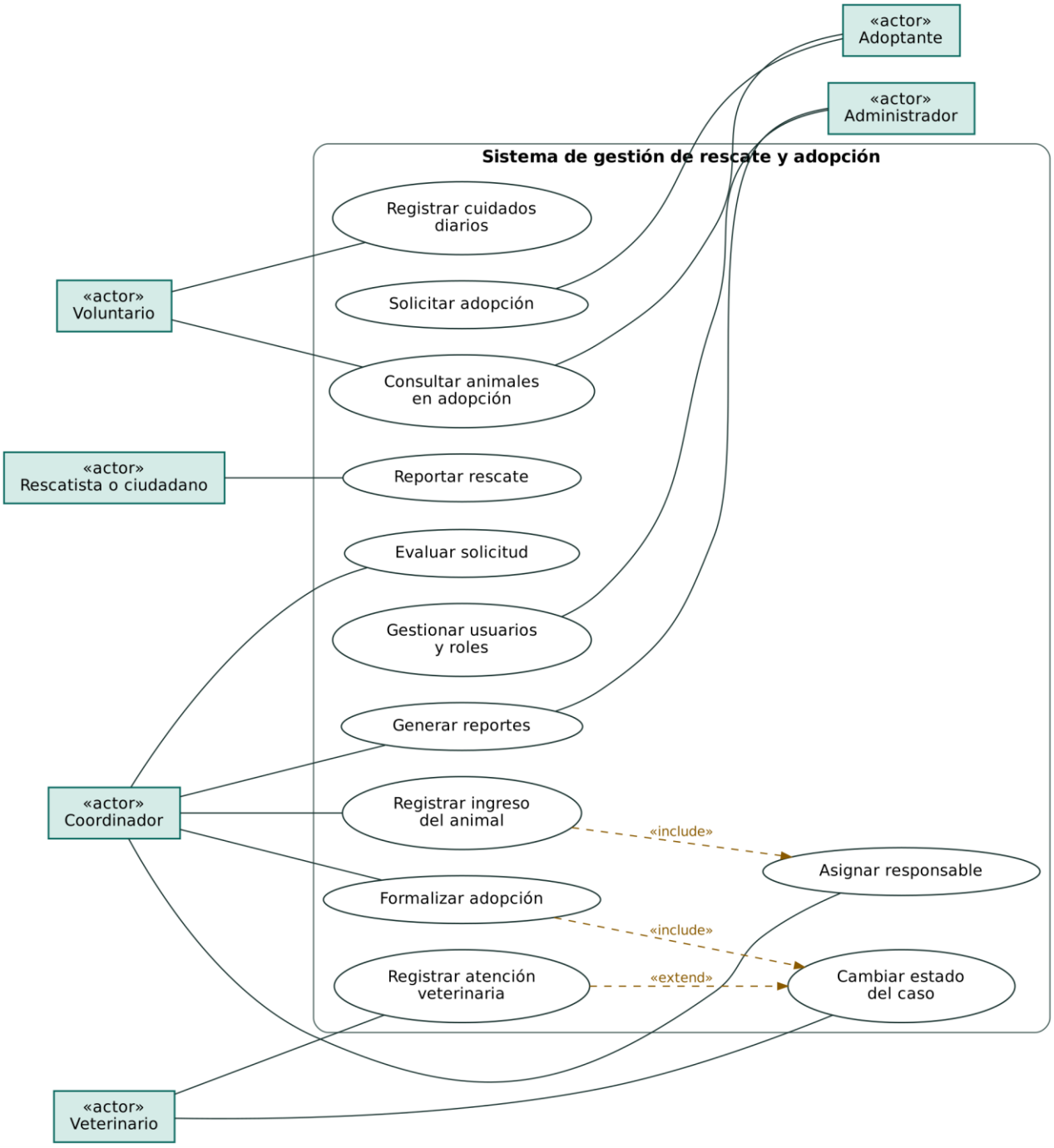
AGOSTO 2026 – ENERO 2027

Contenido

1. Diagrama de casos de uso 2
2. Estados del caso 3
3. Diagrama de clases: animales y estados 4
4. Diagrama de clases: casos y personas 5
5. Diagrama de clases: servicios y puertos 6
6. Principios SOLID aplicados 7
7. Arquitectura general del sistema 8
8. Flujo de versiones y ramas 9
9. Canalización de integración y despliegue 10

1. Diagrama de casos de uso

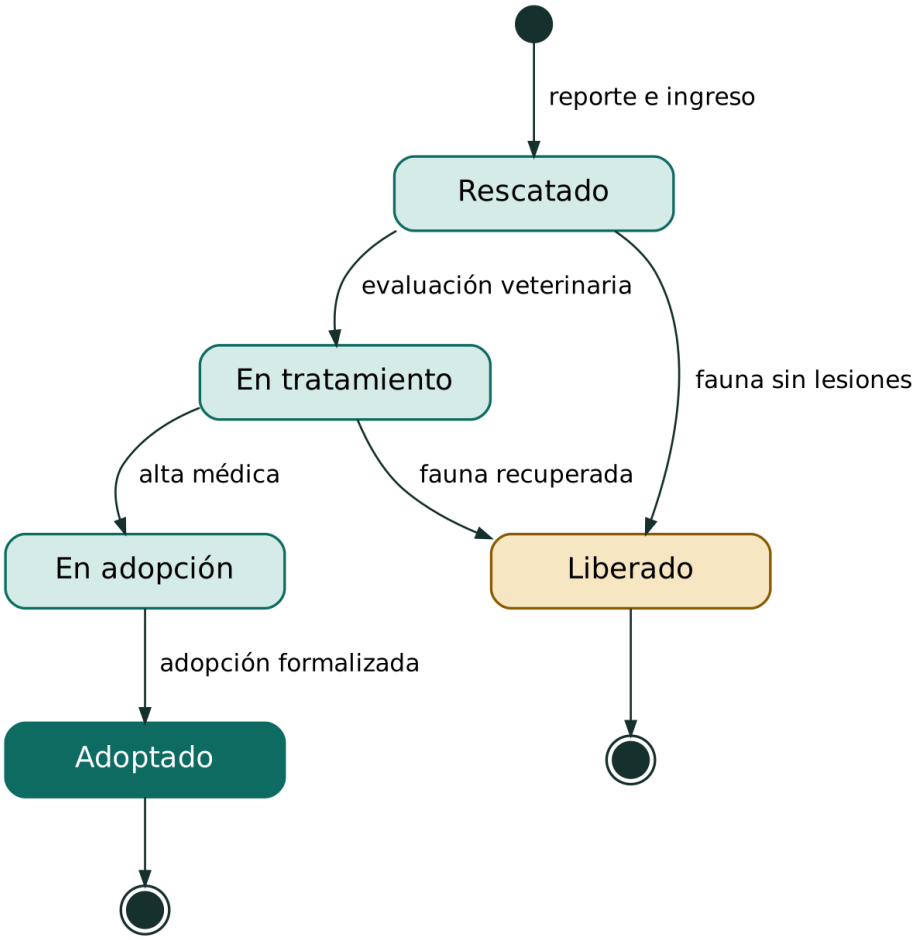
Seis actores interactúan con el sistema. Quien reporta un rescate puede ser un ciudadano sin cuenta; el resto del personal se autentica con un rol que limita lo que puede cambiar en un caso.



Las líneas punteadas son relaciones entre casos de uso: registrar un ingreso incluye asignar un responsable, formalizar una adopción incluye cambiar el estado del caso, y una atención veterinaria puede extenderse a un cambio de estado.

2. Estados del caso

Cada ingreso abre un caso que avanza por estados definidos. Los perros y gatos terminan en adopción; la fauna silvestre puede terminar liberada.

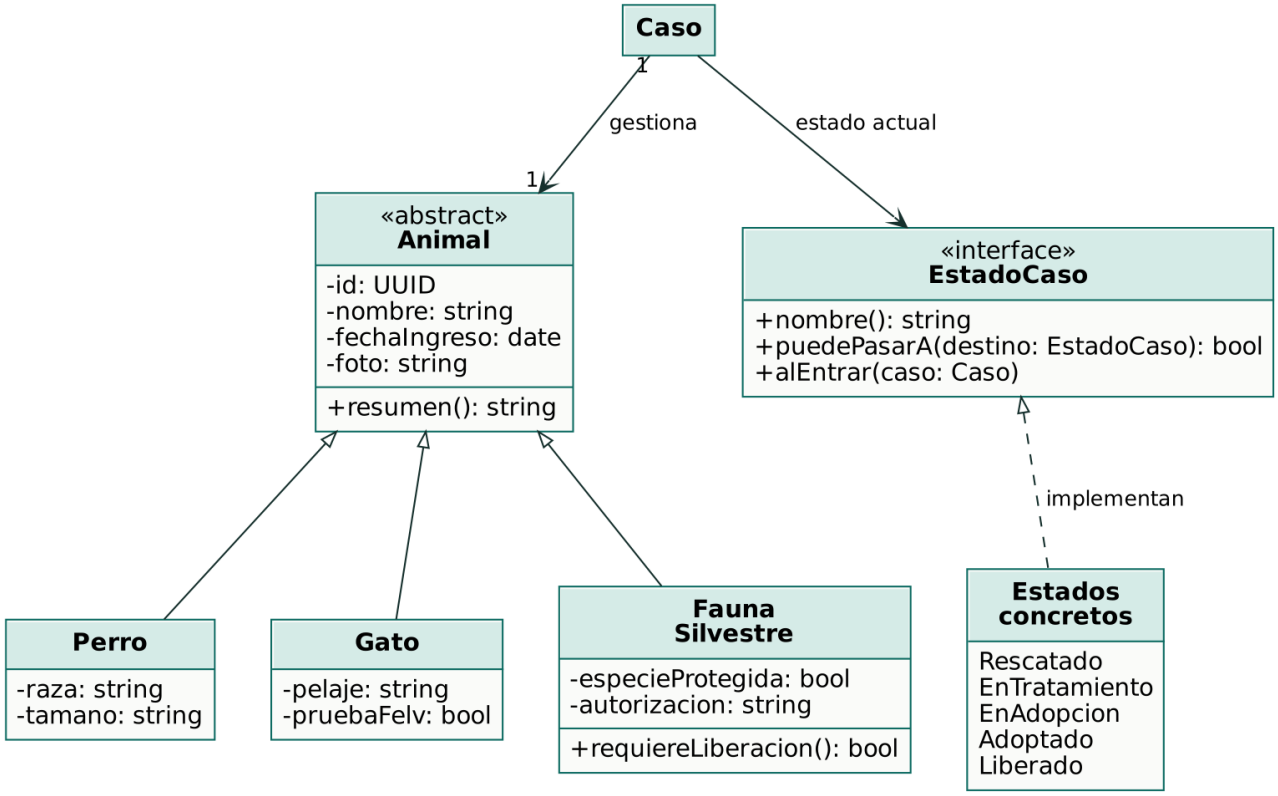


Quién ejecuta cada transición

Transición	Quién la ejecuta	Condición
Rescatado → En tratamiento	Veterinario	Evaluación clínica registrada
En tratamiento → En adopción	Veterinario	Alta médica y animal apto para convivir
En adopción → Adoptado	Coordinador	Solicitud aprobada y contrato firmado
Rescatado o En tratamiento → Liberado	Veterinario y Coordinador	Solo fauna silvestre, con autorización ambiental

3. Diagrama de clases: animales y estados

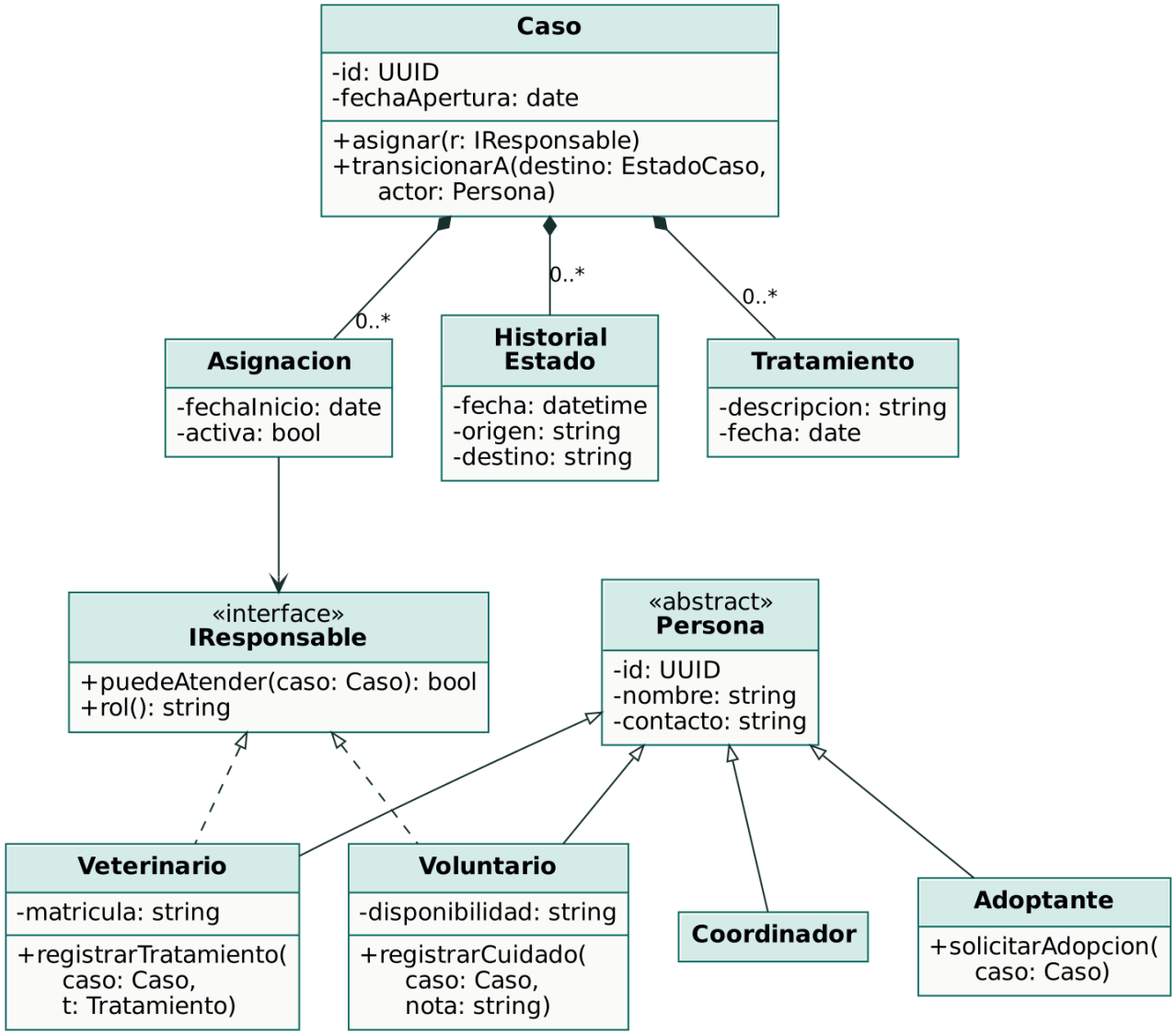
Cada caso gestiona un animal y mantiene un estado actual. Perro, gato y fauna silvestre heredan de Animal; cada estado concreto implementa EstadoCaso (patrón State), de modo que las reglas de transición viven en el estado y no en Caso.



Los estados concretos son Rescatado, EnTratamiento, EnAdopcion, Adoptado y Liberado. Agregar un estado nuevo no obliga a modificar Caso.

4. Diagrama de clases: casos y personas

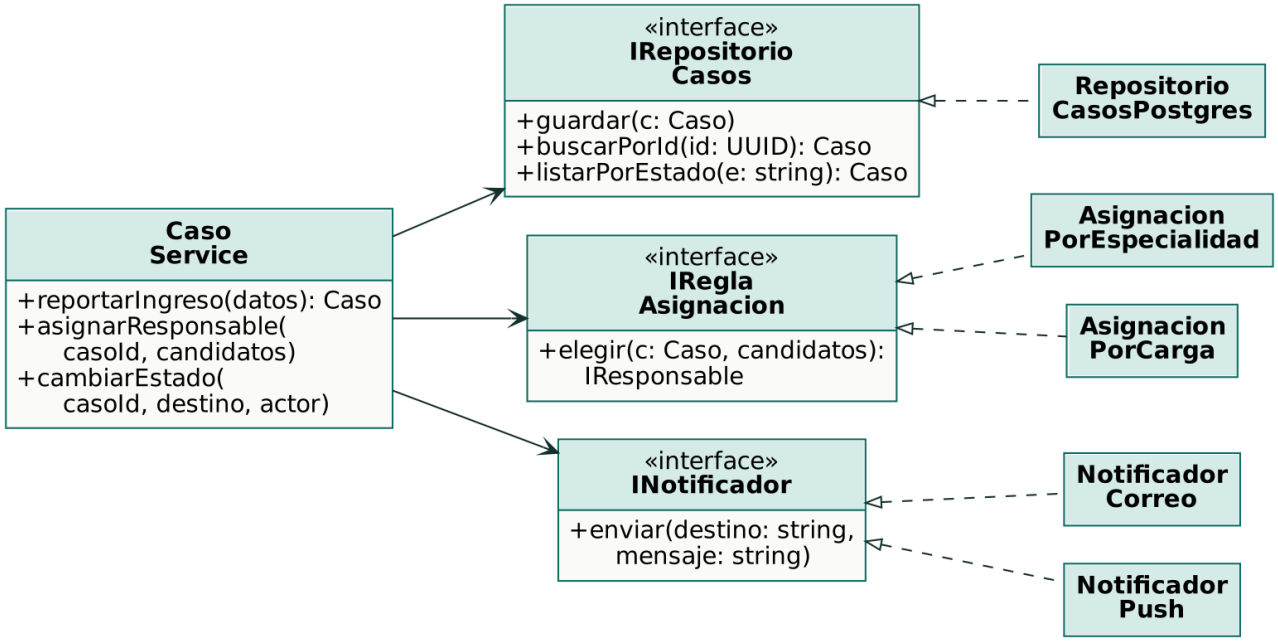
Un caso agrupa sus asignaciones, su historial de estados y sus tratamientos. Solo quienes implementan IResponsable pueden recibir una asignación.



El adoptante es una Persona pero no un IResponsable, así que nunca puede recibir la asignación de un caso.

5. Diagrama de clases: servicios y puertos

CasoService orquesta los casos de uso y depende solo de interfaces. PostgreSQL, el correo y el push son implementaciones intercambiables.



Las reglas de asignación (IReglaAsignacion) se pueden sumar o reemplazar sin modificar el servicio.

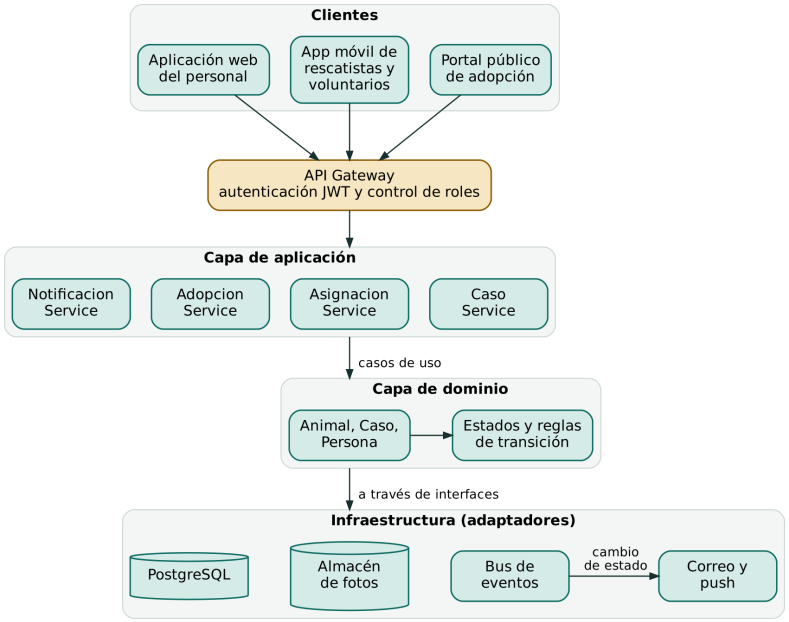
6. Principios SOLID aplicados

Dónde aparece cada principio en el diseño de clases.

Principio	Aplicación en el diseño
S · Responsabilidad única	Caso controla su ciclo de vida, HistorialEstado guarda la auditoría y CasoService orquesta. Ninguna clase mezcla reglas de negocio con acceso a datos o envío de mensajes.
O · Abierto/cerrado	Un nuevo animal (por ejemplo Ave), un nuevo estado (EnCuarentena) o una nueva regla de asignación se agregan como clases nuevas, sin modificar Caso ni CasoService.
L · Sustitución de Liskov	Perro, Gato y FaunaSilvestre se usan donde se espera un Animal, y cualquier estado concreto reemplaza a EstadoCaso sin romper el flujo.
I · Segregación de interfaces	IResponsable, IRepositorioCasos, IReglaAsignacion e INotificador son pequeñas y específicas. El adoptante no queda obligado a implementar operaciones de atención.
D · Inversión de dependencias	CasoService depende de interfaces. PostgreSQL, el correo y el push son detalles que se inyectan al arrancar la aplicación.

7. Arquitectura general del sistema

Arquitectura por capas con dependencias hacia el dominio. Se despliega como un monolito modular, lo que mantiene bajo el costo operativo de un refugio y deja abierta la separación posterior de módulos como adopciones o notificaciones.



Los servicios de aplicación llegan a PostgreSQL, a las fotos y al correo a través de las interfaces de la vista de servicios. Cada cambio de estado publica un evento que dispara las notificaciones al responsable y, en adopciones, al adoptante.

El diseño usa los cuatro patrones pedidos y dos más que ya aparecían en los diagramas de clases. Cada uno responde a un problema concreto del sistema de rescate y adopción.

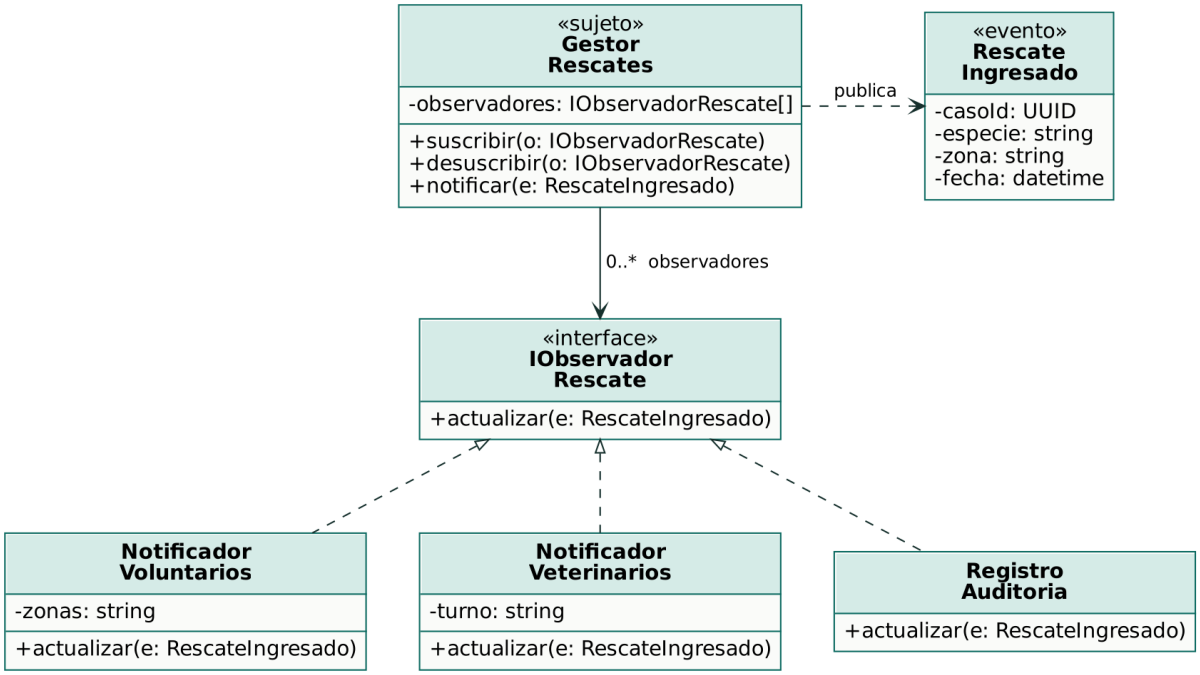
Patrón	***Dónde se aplica***	***Problema del refugio que resuelve***
Observer	*GestorRescates y sus observadores: voluntarios, veterinarios y auditoría*	*Avisar al instante a los voluntarios cuando entra un rescate nuevo, sin que CasoService conozca a cada receptor.*
Factory Method	*ExpedienteFactory y un creador por especie*	*Crear un expediente distinto para perro, gato o fauna silvestre sin condicionales por especie repartidos en el código.*
Singleton	*MotorGestionCasos y ConexionBaseDatos*	*Garantizar un solo motor de casos y un solo pool de conexiones, para que dos instancias no asignen el mismo caso ni agoten las conexiones.*
MVC	*Vistas web y móvil, controladores REST y modelo de dominio*	*Separar la interfaz de la lógica de negocio para que el portal, la app y el panel del personal usen las mismas reglas.*

Patrón	*Dónde se aplica*	*Problema del refugio que resuelve*
State (adicional)	*EstadoCaso y sus estados concretos*	*Controlar qué transiciones son válidas: Rescatado, En tratamiento, En adopción, Adoptado o Liberado.*
Strategy (adicional)	*IReglaAsignacion y sus reglas*	*Cambiar el criterio para elegir responsable, por especialidad o por carga, sin modificar el servicio.*

State y Strategy se muestran en los diagramas de las hojas 4 y 6. Las hojas siguientes explican los cuatro patrones pedidos, uno por hoja.

8. Patrón Observer: notificaciones de rescates

Propósito. Definir una dependencia de uno a muchos: cuando un objeto cambia, todos los interesados se enteran automáticamente.



Problema en el sistema

Cuando alguien reporta un rescate, deben enterarse los voluntarios de la zona, el veterinario de turno y el registro de auditoría. Si CasoService llamara a cada uno, tendría que conocerlos y modificarse cada vez que aparezca un receptor nuevo, por ejemplo un aviso por mensajería.

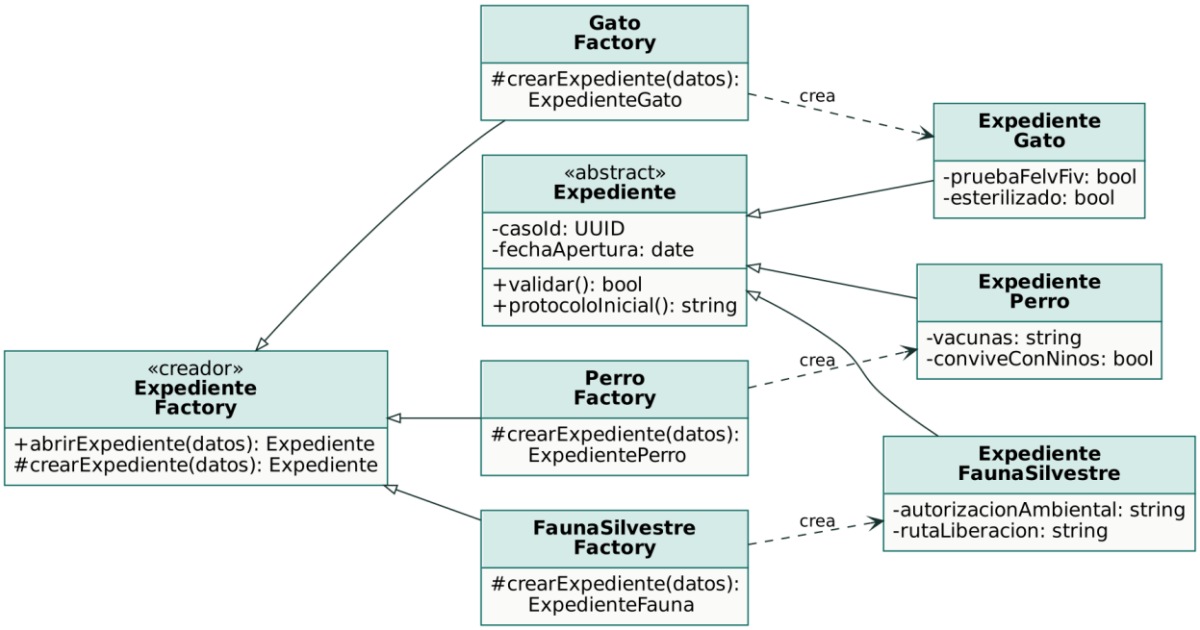
Cómo lo resuelve

GestorRescates guarda una lista de IObservadorRescate. Al registrar un ingreso publica un RescateIngresado y cada observador reacciona por su cuenta. Los observadores se suscriben o se retiran mientras el sistema corre, sin tocar al sujeto. Es el mismo modelo de la programación reactiva: el evento es un flujo de datos y los observadores son suscriptores asíncronos, por lo que el sujeto puede construirse sobre una librería reactiva como RxJS o Project Reactor.

Rol en el patrón	Clases del sistema
Sujeto	GestorRescates
Observador	IObservadorRescate
Observadores concretos	NotificadorVoluntarios, NotificadorVeterinarios, RegistroAuditoria
Evento	RescateIngresado

9. Patrón Factory Method: expedientes por especie

Propósito. Definir una operación para crear objetos y dejar que las subclases decidan qué clase concreta instanciar.



Problema en el sistema

Cada especie necesita datos y validaciones distintas. El perro lleva vacunas y convivencia con niños, el gato la prueba de FeLV y FIV y su esterilización, y la fauna silvestre una autorización ambiental y una ruta de liberación. Sin el patrón, el servicio tendría condicionales por especie repartidos en varios métodos.

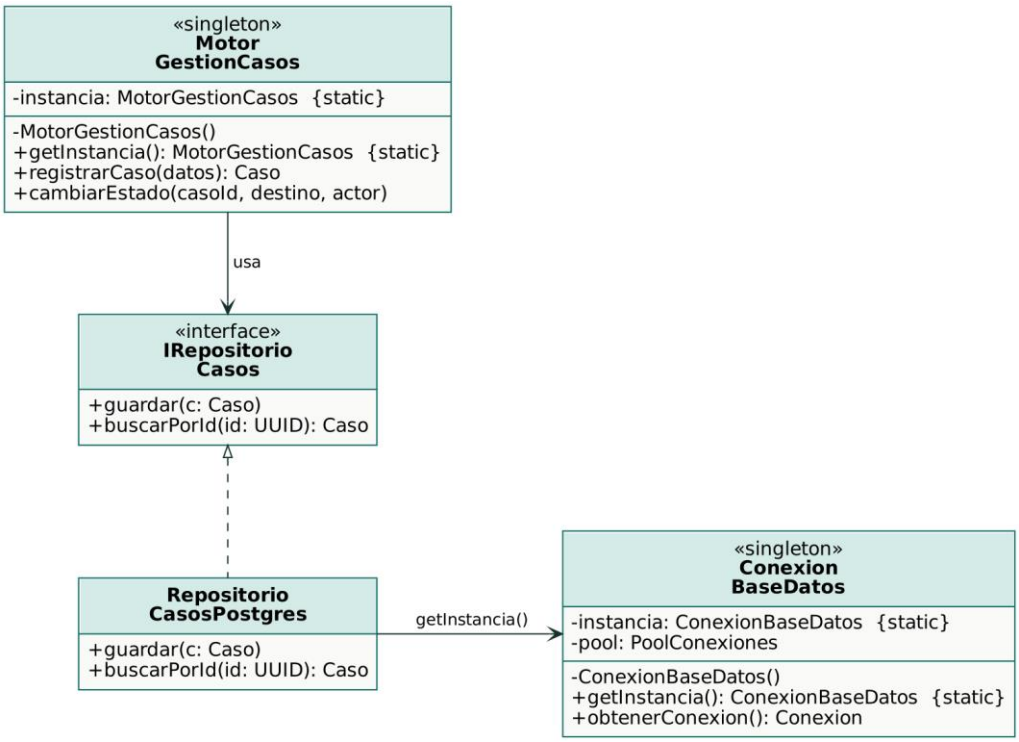
Cómo lo resuelve

ExpedienteFactory define abrirExpediente(datos), que valida y guarda el expediente, y deja en crearExpediente(datos) la creación del objeto. PerroFactory, GatoFactory y FaunaSilvestreFactory sobrescriben ese método. El servicio elige el creador según la especie del reporte y trabaja siempre con el tipo Expediente. Para sumar una especie nueva, como un ave, basta una fábrica y un expediente nuevos, sin modificar el código existente.

Rol en el patrón	Clases del sistema
Creador	ExpedienteFactory
Creadores concretos	PerroFactory, GatoFactory, FaunaSilvestreFactory
Producto	Expediente
Productos concretos	ExpedientePerro, ExpedienteGato, ExpedienteFaunaSilvestre

10. Patrón Singleton: motor de casos y conexión a datos

Propósito. Garantizar que una clase tenga una sola instancia y ofrecer un punto de acceso a ella.



Problema en el sistema

Con dos motores de gestión de casos, dos coordinadores podrían asignar el mismo caso a voluntarios distintos. Además, cada instancia abriría sus propias conexiones a PostgreSQL hasta agotar las disponibles.

Cómo lo resuelve

MotorGestionCasos y ConexionBaseDatos tienen constructor privado y un método estático getInstancia() que crea el objeto la primera vez y después devuelve siempre el mismo. La conexión administra un pool y su creación debe ser segura entre hilos. MotorGestionCasos es la instancia central que ejecuta lo que la vista de servicios llama CasoService.

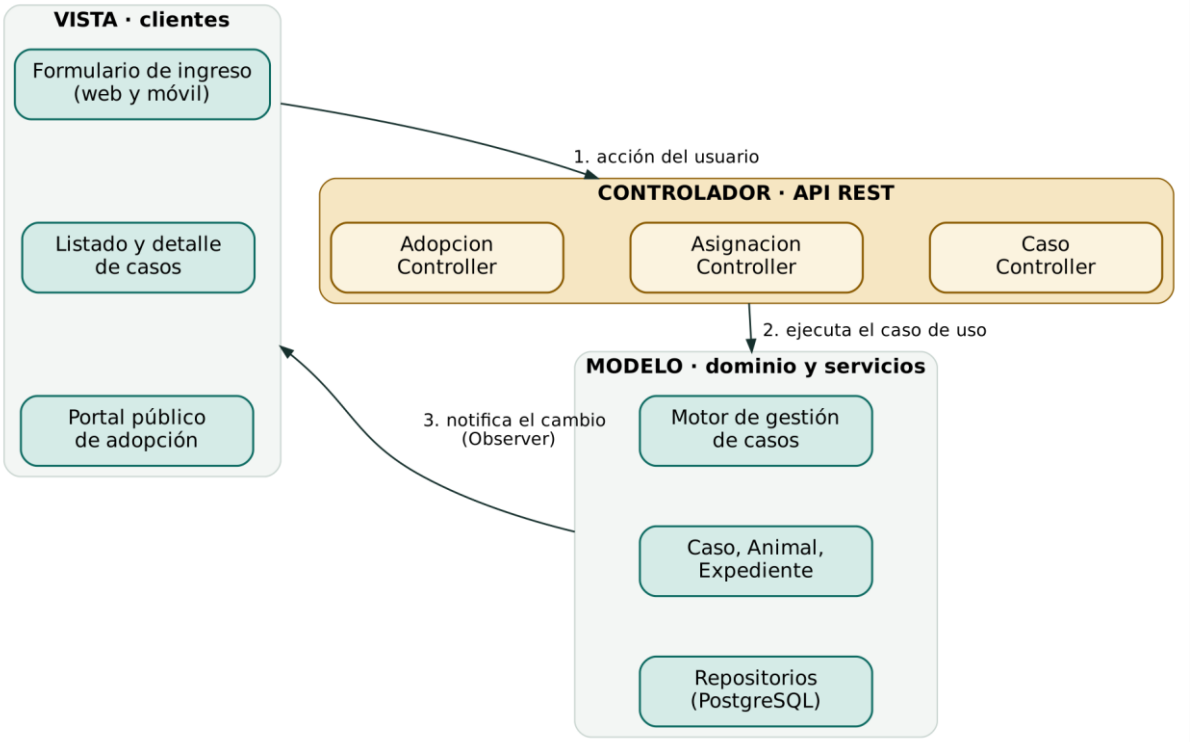
Precauciones

Un Singleton introduce estado global y dificulta las pruebas, así que el motor depende de la

interfaz IRepositorioCasos y solo RepositorioCasosPostgres accede a ConexionBaseDatos. En las pruebas se sustituye por un repositorio en memoria. Además, la unicidad es por proceso: si la aplicación se replica en varios servidores, la asignación única se refuerza con restricciones en la base de datos.

11. Patrón MVC: organización de la aplicación

Propósito. Separar la interfaz (vista), la lógica de entrada (controlador) y la lógica de negocio (modelo).



Problema en el sistema

El personal usa la aplicación web, los voluntarios la app móvil y los adoptantes el portal público. Si cada interfaz incluyera reglas de negocio, habría que duplicarlas y corregirlas en tres lugares.

Cómo lo resuelve

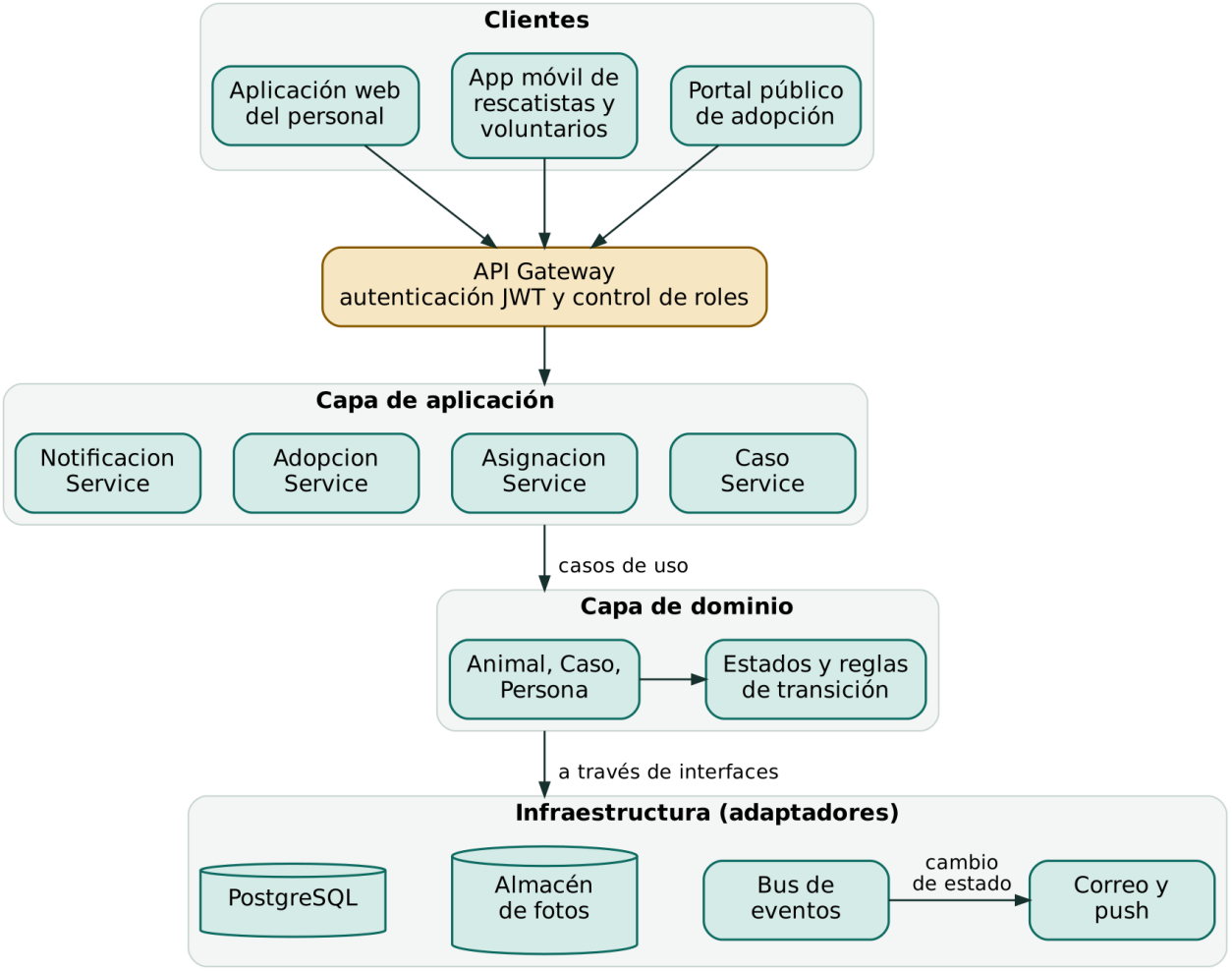
Las vistas solo muestran datos y capturan acciones. Los controladores REST reciben la petición, validan la entrada y llaman al caso de uso. El modelo contiene las reglas y, cuando cambia un estado, publica un evento (Observer) con el que las vistas se actualizan.

Capa MVC	Clases del sistema	Capa de la arquitectura
Vista	Formulario de ingreso, listado de casos, portal público de adopción	Clientes
Controlador	CasoController, AsignacionController, AdopcionController	API y gateway

Capa MVC	Clases del sistema	Capa de la arquitectura
Modelo	Animal, Caso, Expediente, MotorGestionCasos, repositorios	Aplicación, dominio e infraestructura

12. Arquitectura general del sistema

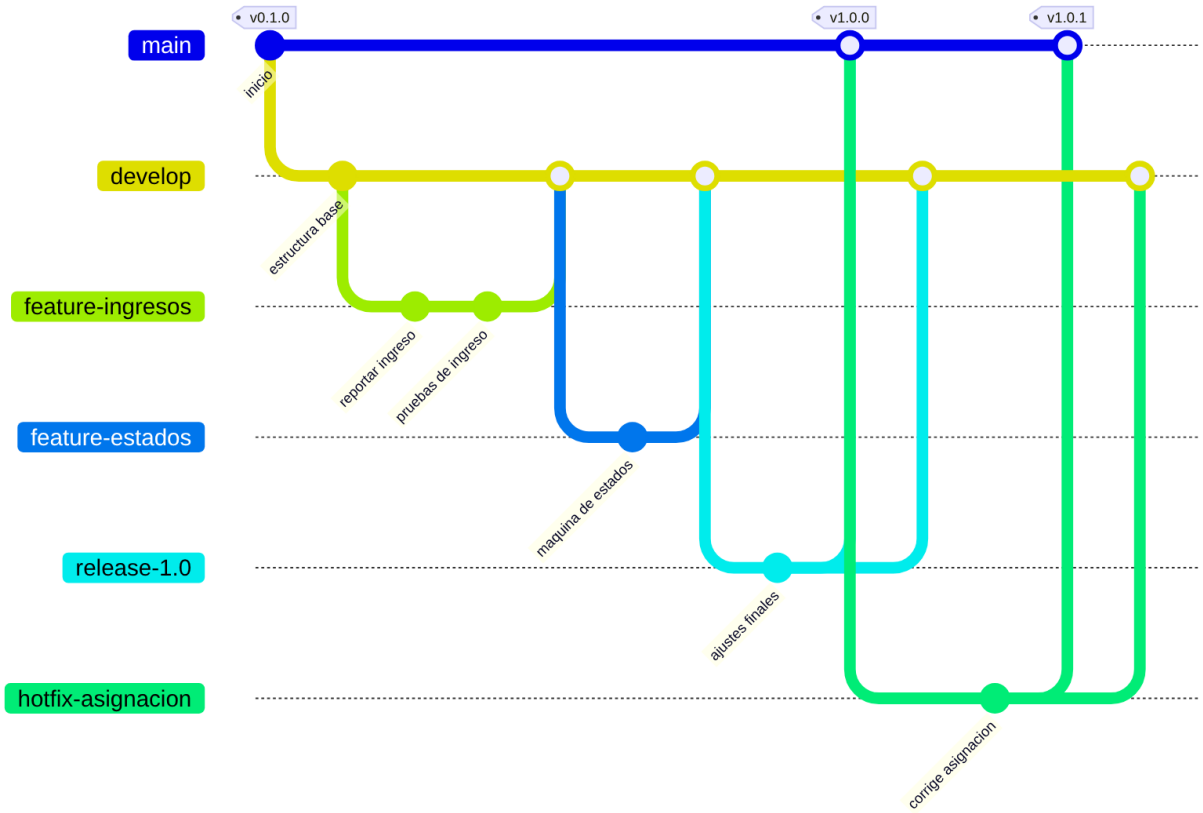
Arquitectura por capas con dependencias hacia el dominio. Se despliega como un monolito modular, lo que mantiene bajo el costo operativo de un refugio y deja abierta la separación posterior de módulos como adopciones o notificaciones.



Los servicios de aplicación llegan a PostgreSQL, a las fotos y al correo a través de las interfaces de la vista de servicios. Cada cambio de estado publica un evento que dispara las notificaciones al responsable y, en adopciones, al adoptante.

13. Flujo de versiones y ramas

El código se versiona con Git Flow. Los números de versión siguen el esquema MAYOR.MENOR.PARCHE.

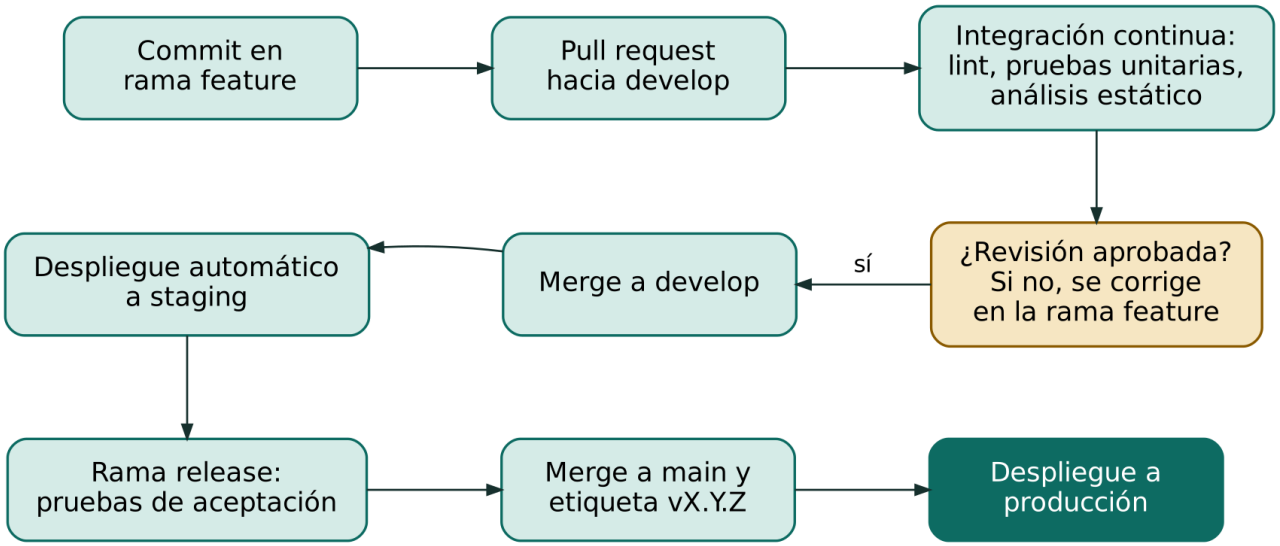


Ramas y reglas

Rama	Uso	Reglas
main	Código en producción, cada merge lleva una etiqueta de versión	Protegida, solo recibe merges de release y hotfix
develop	Integración continua del trabajo en curso	Protegida, requiere pull request con una revisión
feature-*	Una funcionalidad por rama, por ejemplo feature-ingresos	Nace de develop y vuelve a develop
release-*	Estabilización antes de publicar	Solo correcciones, sin funcionalidades nuevas
hotfix-*	Corrección urgente en producción	Nace de main, se fusiona en main y en develop

14. Canalización de integración y despliegue

Cada cambio recorre la misma secuencia: revisión, pruebas automáticas, staging y producción. Solo el código fusionado en main con etiqueta de versión llega a producción.



15. Justificación de Gitflow

Gitflow se adapta a este proyecto porque combina entregas planificadas, trabajo en paralelo de varias personas y un sistema que no puede fallar mientras hay animales en tratamiento.

Necesidad del proyecto	Cómo la cubre Gitflow
Varias personas desarrollan funciones distintas a la vez: ingresos, estados, notificaciones	Una rama feature-* por función, aislada del resto hasta que se revisa y se integra en develop.
La versión en producción debe ser estable, porque el personal registra tratamientos y adopciones	main solo recibe código probado desde release-* o hotfix-*, así que siempre se puede desplegar.
Coordinadores y veterinarios deben probar el sistema antes de publicarlo	La rama release-* congela las funciones y permite pruebas de aceptación y correcciones menores.
Un error en producción no puede esperar al siguiente lanzamiento	hotfix-* nace de main, se corrige y se publica como v1.0.1 sin arrastrar funciones a medio terminar.
Hay que saber qué versión está en uso y poder volver a una anterior	Cada merge a main lleva una etiqueta de versión, por ejemplo v1.0.0 y v1.0.1.
El código debe revisarse antes de integrarse	Los pull requests hacia develop exigen una revisión y pasan por la integración continua.

Frente a otras estrategias

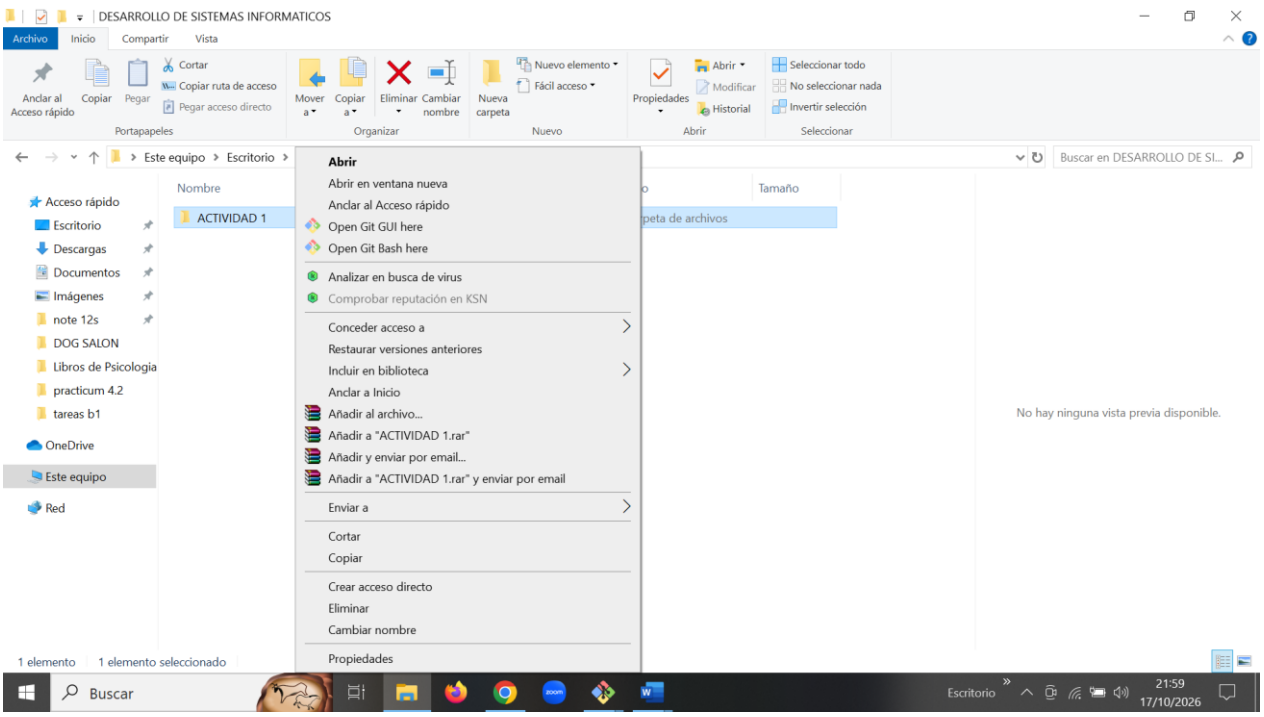
Estrategia	Ajuste al proyecto
GitHub Flow	Más simple y buena para desplegar de forma continua, pero no tiene rama de estabilización ni etiquetas de versión por defecto. Se ajusta peor a lanzamientos planificados con pruebas de aceptación.

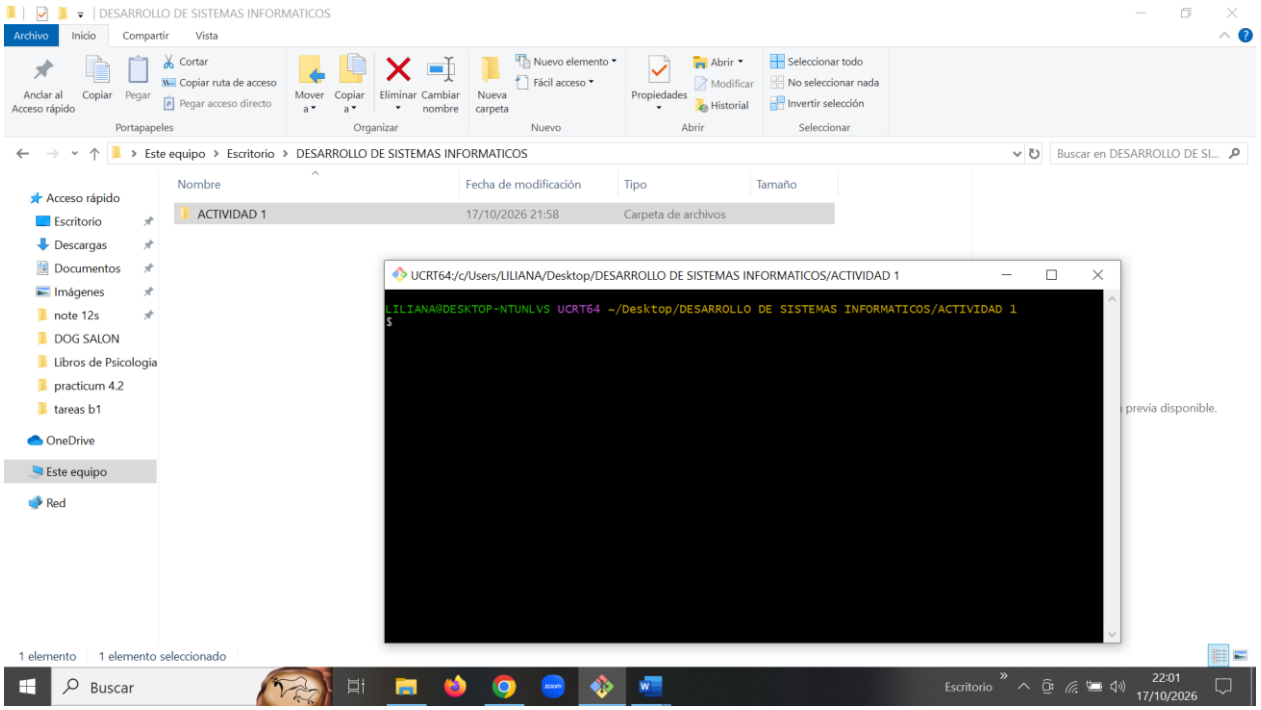
Estrategia	Ajuste al proyecto
Trunk-based	Exige pruebas automáticas muy completas y banderas de funciones. Es demasiada exigencia para un equipo pequeño que apenas arranca.
Gitflow	Una rama definida para cada tarea y versiones etiquetadas. A cambio, hay más ramas que gestionar.

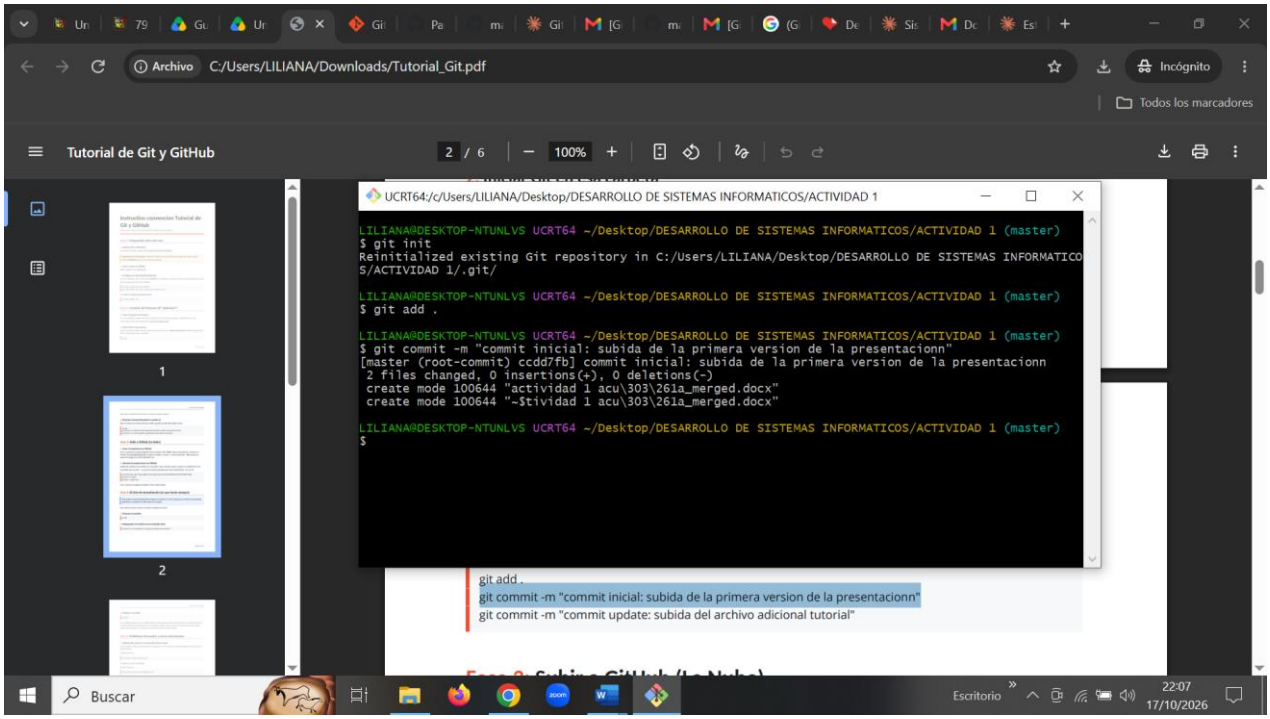
Costo y cómo reducirlo

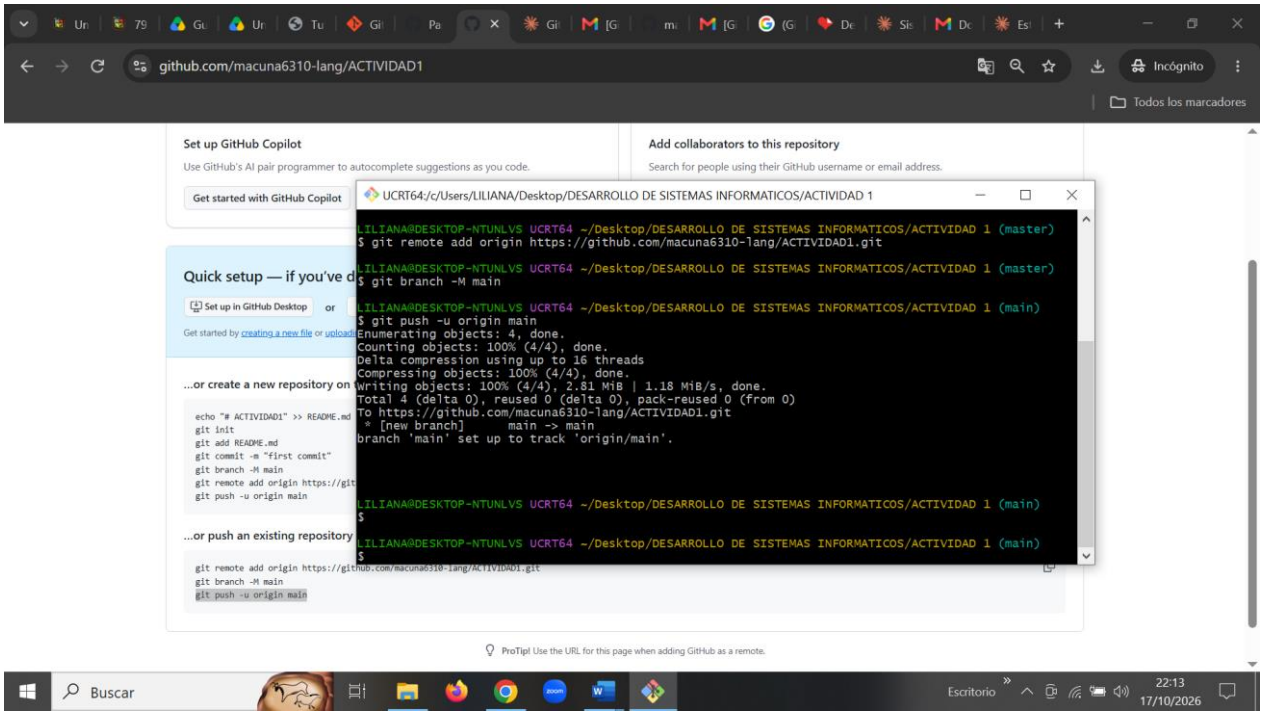
Gitflow agrega ramas y pasos de integración. Se reduce con ramas feature de corta vida, borrando cada rama tras el merge, y con una canalización automática que ejecute las pruebas en cada pull request.

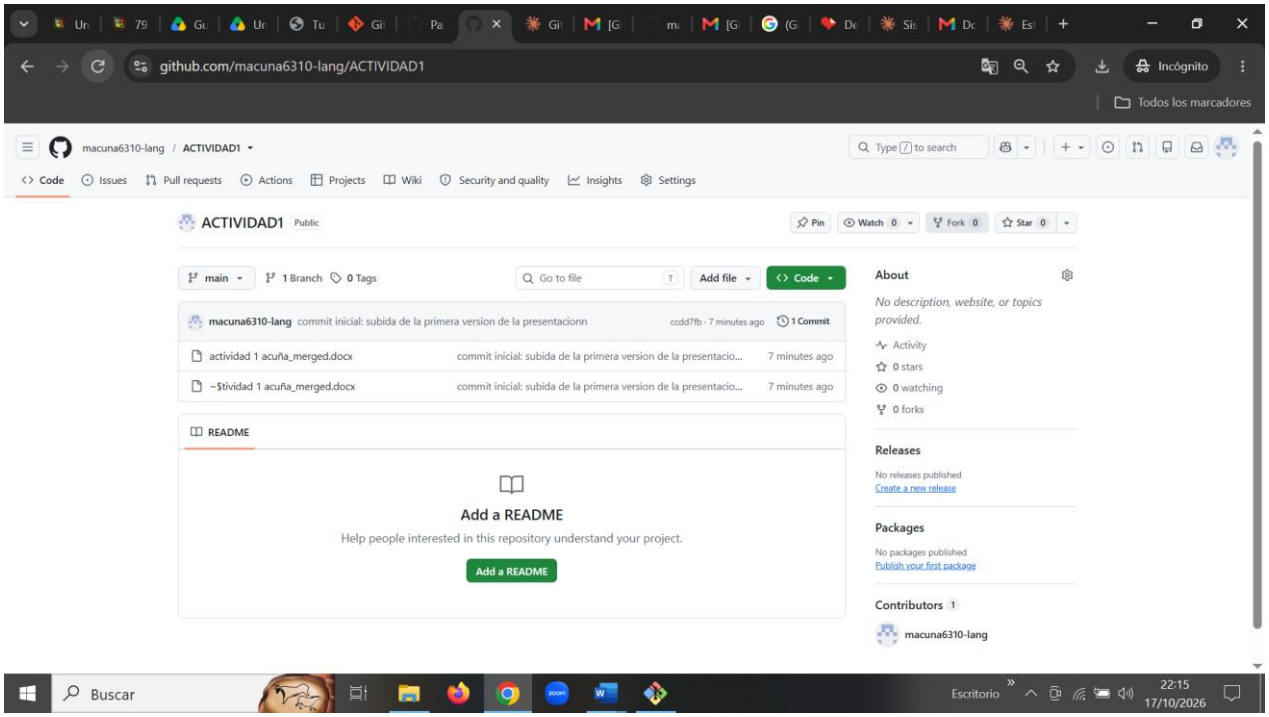
GitHub




DESARROLLO DE SISTEMAS INFORMATICOS
Archivo
Inicio
Compartir
Vista
Este equipo › Escritorio › DESARROLLO DE SISTEMAS INFORMATICOS
Buscar en DESARROLLO DE SI...
Acceso rápido
Escritorio
Descargas
Documentos
Imágenes
note 12s
DOG SALON
Libros de Psicologia
practicum 4.2
tareas b1
OneDrive
Este equipo
Red
Nombre
Fecha de modificación
Tipo
Tamaño
ACTIVIDAD 1
17/10/2026 21:58
Carpeta de archivos
UCRT64:/c/Users/LILIANA/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/ACTIVIDAD 1
LILIANA@DESKTOP-NTUNLVS UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/ACTIVIDAD 1
$
1 elemento
1 elemento seleccionado
Buscar
Escritorio
22:01
17/10/2026


Archivo
C:/Users/LILIANA/Downloads/Tutorial_Git.pdf
Incógnito
Todos los marcadores
Tutorial de Git y GitHub
2 / 6
100%
1
2
UCRT64:/c/Users/LILIANA/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/ACTIVIDAD 1
LILIANA@DESKTOP-NTUNLVS UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/ACTIVIDAD 1 (master)
$ git init
Reinitialized existing Git repository in C:/Users/LILIANA/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/ACTIVIDAD 1/.git/
LILIANA@DESKTOP-NTUNLVS UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/ACTIVIDAD 1 (master)
$ git add .
LILIANA@DESKTOP-NTUNLVS UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/ACTIVIDAD 1 (master)
$ git commit -m "commit inicial: subida de la primera version de la presentacionn"
[master (root-commit) ccdd7fb] commit inicial: subida de la primera version de la presentacionn
2 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 "actividad 1 acu\303\261a_merged.docx"
create mode 100644 "~$tividad 1 acu\303\261a_merged.docx"
LILIANA@DESKTOP-NTUNLVS UCRT64 ~/Desktop/DESARROLLO DE SISTEMAS INFORMATICOS/ACTIVIDAD 1 (master)
$
git add .
git commit -m "commit inicial: subida de la primera version de la presentacionn"
git commit -m "commit update: subida del archivo adicional tutorial"
Buscar
Escritorio
22:07
17/10/2026





Link

https://github.com/macuna6310-lang/ACTIVIDAD1